

Reviewer Pack — Minimal Frobenius-Schur Indicators and Resolution Proof Width...

Entry 2d5f55632128 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 01:15:26 UTC

Minimal Frobenius-Schur Indicators and Resolution Proof Width Bound

Entry ID: 2d5f55632128 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 01:15:26 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every CNF φ with n variables, the minimal Frobenius-Schur indicator (FSI) of the corresponding representation $R(\varphi)$ is linearly correlated with its resolution proof width $w(\varphi)$, such that $\text{FSI}(R(\varphi)) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Representation theory often provides invariants for algebraic structures, and Frobenius-Schur indicators are known to be closely related to the structure of representations. These invariants could potentially expose deeper connections between representation theory and computational complexity, specifically providing a new perspective on proof width.

Taxonomy category: resolution_proof_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b3dc0023bb9829bd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every CNF φ with n variables, if the correlation coefficient between $\text{FSI}(R(\varphi))$ and $w(\varphi)$ is ≥ 0.8 and the mean absolute difference between $\text{FSI}(R(\varphi))$ and $w(\varphi)$ across all seeds is ≤ 3 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Frobenius-Schur indicator" AND "resolution proof width" - "representation theory" AND "resolution complexity" AND "Frobenius-Schur indicator" - "minimal Frobenius-Schur indicator" IN TITLE AND resolution proof width

Top relevant hits considered: - [http://arxiv.org/abs/1902.08221v2] Compact minimal hypersurfaces of index one and the width of real projective spaces - [http://arxiv.org/abs/math/0101208v3] A Proof of Desingularization over fields of characteristic zero - [http://arxiv.org/abs/1810.10111v2] A Simple Constructive Proof of Wigner’s Theorem

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def dpll(cnf, assignment=[]):
        if not cnf:
            return True
        literal = next((l for l in range(1, len(cnf) + 1) if l not in assignment and -l not in assignment))
        if literal is None:
            return False

        def propagate(lit):
            new_cnf = []
            for clause in cnf:
                if lit in clause:
                    continue
                elif -lit in clause:
                    clause.remove(-lit)
                if not clause:
                    return False
            return new_cnf

        return dpll(propagate(literal), assignment + [literal])
```

```

        else:
            new_cnf.append(clause)
        return new_cnf

def backtrack(lit):
    assignment.pop()

    if dpll(propagate(literal), assignment + [literal]):
        return True
    backtrack(literal)

    if dpll(propagate(-literal), assignment + [-literal]):
        return True
    backtrack(-literal)

    return False

def frobenius_schur_indicator(matrix):
    n = len(matrix)
    trace = sum(matrix[i][i] for i in range(n))
    det = 1.0
    for row in matrix:
        det *= abs(sum(row[j] * matrix[j][i] for j in range(n)))
    return trace / det

def resolution_width(cnf):
    queue = cnf[:]
    while True:
        new_clause = None
        for i in range(len(queue)):
            for j in range(i + 1, len(queue)):
                if set(queue[i]).intersection(set(queue[j])):
                    new_clause = [l for l in queue[i] if l not in queue[j]] + [l for l in queue[j] if l not in queue[i]]
                    break
            if new_clause:
                break
        if not new_clause:
            return len(queue)
        queue.append(new_clause)

n_max = 0
instances_tested = 0
total_fsi = 0.0
total_diff = 0.0

for n in [5, 10, 15, 20, 30, 40]:
    if n > n_max:
        n_max = n

    for _ in range(5):
        cnf = generate_cnf(n)
        instances_tested += 1

        matrix = [[0] * (n + 1) for _ in range(n + 1)]
        for clause in cnf:
            for l in clause:

```

```

        if l > 0:
            matrix[l][l] += 1
        else:
            matrix[-l][-l] -= 1

    fsi = frobenius_schur_indicator(matrix)
    width = resolution_width(cnf)

    total_fsi += fsi
    total_diff += abs(fsi - width)

mean_fsi = total_fsi / instances_tested
mean_diff = total_diff / instances_tested

correlation_coefficient = (instances_tested * sum(fsi * width for fsi, width in zip([mean_fsi]
conjecture_holds = correlation_coefficient >= 0.8 and mean_diff <= 3

return {
    "metric_name": "Frobenius-Schur Indicator vs Resolution Width",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else "Correlation coefficient < 0.8 or mean absolute
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Correlation coefficient < 0.8 or mean absolute

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6fbaab6d.py", line 133, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6fbaab6d.py", line 105, in run_trial
    fsi = frobenius_schur_indicator(matrix)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6fbaab6d.py", line 66, in frobenius_schur
    det *= abs(sum(row[j] * matrix[j][i] for j in range(n)))
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6fbaab6d.py", line 66, in <genexpr>
    det *= abs(sum(row[j] * matrix[j][i] for j in range(n)))
                    ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to an undefined variable 'i', which prevented it from producing any data to evaluate the conjecture. | next: Debug the error in the test code and rerun the test to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12819
2	preregistration	ollama_remote	glm4:latest	0	0	9595
3	novelty	ollama_remote	glm4:latest	0	0	8515
4	novelty	ollama_remote	glm4:latest	0	0	9481
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	66800
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11967
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17285
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15151
9	judge	ollama_remote	glm4:latest	0	0	27429

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 179043 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/2d5f55632128.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2d5f55632128.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2d5f55632128.tar.gz` (if generated)